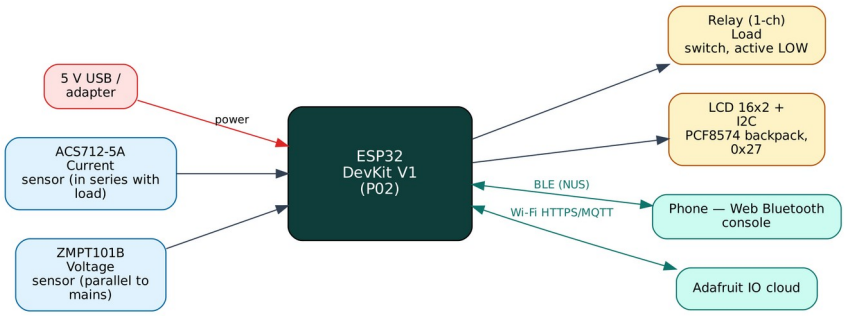
A PROJECT REPORT ON

Smart Energy Monitor with Overload Protection

IoT project using ESP32 with Bluetooth Low Energy and Wi-Fi (Adafruit IO)



Project: P02 · Advanced level

Peripherals: ACS712-5A current sensor, ZMPT101B voltage sensor, 16x2 I2C LCD, 1-channel relay

Submitted by: _____

Under the guidance of: _____

Department / Institution: _____

October 2026

Certificate

This is to certify that the project report entitled **“Smart Energy Monitor with Overload Protection”** is a bonafide record of the work carried out by _____ (Roll No. _____) in partial fulfilment of the requirements for the award of _____ in _____ during the academic year _____.

Project Guide: _____ Head of Department: _____

Date: _____ Place: _____

Acknowledgement

I express my sincere gratitude to my project guide _____ for continuous guidance and support, to the Head of the Department _____, and to all faculty members and friends who helped me complete this project successfully.

Contents

Abstract 6

1. Introduction 7

1.1 Problem Statement 7
1.2 Objectives 7
1.3 Scope of the Project 7
1.4 Organisation of the Report 7

2. Existing System and Proposed System 8

2.1 Existing Systems 8
2.2 Proposed System 8
2.3 Why Both BLE and Wi-Fi? 8

3. System Design 9

3.1 Block Diagram 9
3.2 System Architecture 9
3.3 Hardware Components and Cost 9
3.4 ESP32 DevKit V1 10
3.5 Working Principle of Components 10

4. Circuit Design 11

4.1 Circuit Diagram 11
4.2 Pin Connections 11
4.3 Interfacing and Power Notes 12

5. Data Flow Diagrams 13

5.1 DFD Level 0 (Context Diagram) 13
5.2 DFD Level 1 14
5.3 Data Dictionary 15

6. Software Design and Implementation 16

6.1 Development Environment 16
6.2 Firmware Architecture 16
6.3 Working of the System 16
6.4 Flowchart 17
6.5 BLE Communication Protocol 18
6.6 Cloud Integration (Adafruit IO) 18
6.7 Configurable Settings 19

7. Setup and Deployment Guide 20

7.1 Hardware Assembly 20
7.2 Flashing the Firmware 20
7.3 Cloud Setup (Adafruit IO) 20
7.4 BLE App Setup (Web Bluetooth Console) 21
7.5 First-Run Checklist 22

8. Testing and Results 23

8.1 Expected Output 23
8.2 Troubleshooting 23

9. Discussion 24

9.1 Advantages 24
9.2 Limitations 24
9.3 Safety Precautions 24
9.4 Security Considerations 24
9.5 Future Scope 24

10. Conclusion..... 25

References..... 26

Appendix A: Project Source Code..... 27

Appendix B: Shared Library Code..... 31

Abstract

This project measures the electrical consumption of a single AC appliance and protects it from overload. A ZMPT101B transformer module senses the 230 V mains voltage and an ACS712-5A Hall-effect module senses the load current. The ESP32 samples both channels together at 10 kHz for ten mains cycles and computes true RMS voltage, true RMS current, real power, apparent power and power factor, then integrates real power into kilowatt-hours and an estimated electricity cost in rupees. A 16x2 LCD shows the values locally and a relay can disconnect the load. The relay trips automatically when power stays above a set limit. Data is published to Adafruit IO over MQTT, where a dashboard shows gauges and charts and a toggle switches the load remotely. Over BLE the same data, relay control and a two-point calibration routine are available on a phone.

Keywords: ESP32, Internet of Things, Bluetooth Low Energy, Wi-Fi, Adafruit IO, ACS712-5A current sensor, ZMPT101B voltage sensor, 16x2 I2C LCD, 1-channel relay.

1. Introduction

The Internet of Things (IoT) connects physical devices — sensors and actuators — to networks so that data can be collected, analysed and acted on automatically. The ESP32 from Espressif Systems has made IoT affordable: a single ₹350 board combines a dual-core 240 MHz processor, Wi-Fi and Bluetooth Low Energy. This project uses both radios at once: BLE for direct, local interaction with a phone and Wi-Fi for cloud connectivity through Adafruit IO, a free IoT/cloud platform.

1.1 Problem Statement

Households and small shops in India pay slab-based electricity tariffs but have no way to see which appliance uses how much energy. Faulty or overloaded appliances (heaters, motors, irons) waste energy and can cause fires. Commercial smart plugs with energy metering are costly, cloud-locked and often inaccurate at low loads.

1.2 Objectives

1. Measure true RMS voltage and current of an AC load and compute real power and power factor.
2. Accumulate energy (kWh) persistently and estimate cost using a configurable tariff.
3. Switch the load ON/OFF locally (BLE), remotely (Adafruit IO) and automatically on overload.
4. Display live values on a 16x2 LCD without needing a phone.
5. Provide an in-field calibration procedure using a reference multimeter.

1.3 Scope of the Project

Single-phase loads up to about 1 kW (5 A sensor; relay rated 10 A). The design is for education and monitoring, not for billing. Three-phase systems and bidirectional (solar export) metering are out of scope.

1.4 Organisation of the Report

Chapter 2 reviews existing approaches and describes the proposed system. Chapter 3 covers system design and components, Chapter 4 the circuit, Chapter 5 the data flow diagrams, Chapter 6 the software, Chapter 7 the step-by-step setup of hardware, cloud and app, Chapter 8 testing, Chapter 9 discussion and Chapter 10 the conclusion. Complete source code is given in the appendices.

2. Existing System and Proposed System

2.1 Existing Systems

Simple Arduino energy meters usually measure only current and assume a fixed 230 V and unity power factor, which overestimates the energy of inductive loads such as fans and motors. They rarely include protection or remote switching, and smart plugs that do are closed products.

2.2 Proposed System

The proposed system improves on these in the following ways:

- Simultaneous V and I sampling gives real power and power factor, not just V×I.
- Overload protection with a 3-second delay to ignore inrush current.
- kWh counter saved to flash every 5 minutes so it survives power cuts.
- Two-point software calibration from the phone; no code change needed.
- Remote switching from the Adafruit IO dashboard and local switching over BLE.

2.3 Why Both BLE and Wi-Fi?

Wi-Fi gives the device internet access for remote monitoring, history and control from anywhere, but it depends on a router and an internet connection, and the device needs the Wi-Fi password before it can connect. BLE needs no infrastructure: any modern phone can connect directly within about 10-30 m. Using both gives:

- **Provisioning** — Wi-Fi credentials and cloud keys are sent from the phone over BLE and stored in flash; nothing secret is hard-coded or uploaded to GitHub.
- **Resilience** — local monitoring and control keep working when the internet or router is down.
- **Remote access** — the cloud dashboard works from anywhere when the internet is available.
- **Low cost** — no extra hardware; both radios are built into the ESP32.

The ESP32 shares one 2.4 GHz radio between Wi-Fi and Bluetooth by time-division coexistence. For this to work the Wi-Fi modem-sleep mode must stay enabled, which the firmware ensures. NimBLE, a lightweight BLE stack, is used instead of the default Bluedroid stack to leave more RAM for TLS (HTTPS/MQTTS) connections.

3. System Design

3.1 Block Diagram

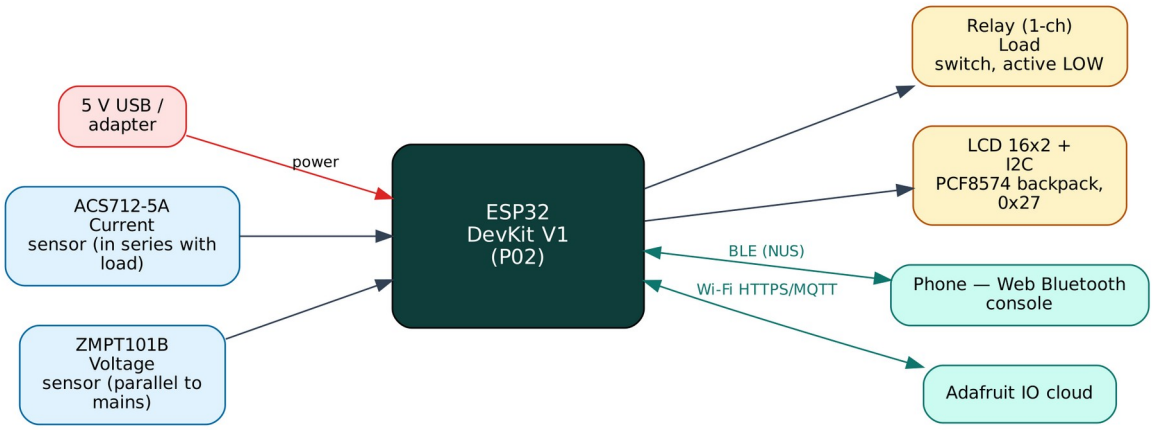


Figure 1: Block diagram of the Smart Energy Monitor

The ESP32 is the central controller. Input devices (blue) feed measurements to it, output devices (amber) are driven by it, and two communication links (green) connect it to the user's phone over BLE and to the Adafruit IO cloud over Wi-Fi. The board is powered from 5 V through its USB connector or VIN pin.

3.2 System Architecture

The system follows the standard three-layer IoT architecture:

- **Perception layer** — ACS712-5A current sensor, ZMPT101B voltage sensor, 16x2 I2C LCD, 1-channel relay connected to the ESP32 GPIO, ADC, I2C, SPI or UART.
- **Network layer** — BLE 4.2 GATT server (Nordic UART Service) and Wi-Fi 802.11 b/g/n with TLS-encrypted MQTT to Adafruit IO.
- **Application layer** — the Web Bluetooth console on the phone and the Adafruit IO dashboard / database.

3.3 Hardware Components and Cost

#	Component	Qty	Specification	Cost
1	ESP32 DevKit V1 (ESP-WROOM-32, 30-pin)	1	Dual-core 240 MHz, Wi-Fi + BLE, 4 MB flash	₹350
2	Breadboard 830-point + jumper wires (M-M, M-F)	1	Prototyping and connections	₹150
3	Micro-USB data cable	1	Power + programming	₹60
4	ACS712-5A current sensor module	1	±5 A, 185 mV/A, Hall-effect, isolated	₹120
5	ZMPT101B AC voltage sensor module	1	0-250 V AC, isolated transformer + op-amp	₹140
6	16x2 LCD with PCF8574 I2C backpack	1	I2C address 0x27 / 0x3F	₹180
7	1-channel 5 V relay module (10 A)	1	Opto-isolated, active-LOW input	₹50
8	Resistors 10 kΩ + 20 kΩ (2 × 10 kΩ)	1	Divider for ACS712 output	₹5

#	Component	Qty	Specification	Cost
9	Fuse holder + 5 A fuse, 2-pin socket, plastic enclosure	1	Mains safety	₹150
	Total (approx.)			**₹1205**

Table 1: Bill of materials with approximate Indian prices (Robu.in / Amazon.in / local market, 2026)

> Prices are indicative and vary by seller. The breadboard, cable and ESP32 can be reused across all ten projects, so the incremental cost of this project is about ₹645.

3.4 ESP32 DevKit V1

Feature	Value
Module	ESP-WROOM-32 (ESP32-D0WD), dual-core Xtensa LX6 up to 240 MHz
Memory	520 KB SRAM, 4 MB SPI flash
Wireless	Wi-Fi 802.11 b/g/n 2.4 GHz; Bluetooth 4.2 BR/EDR + BLE
I/O	25 usable GPIO on the 30-pin board, 12-bit ADC (ADC1: GPIO32-39), 2 × DAC, PWM on any output
Interfaces	3 × UART, 2 × I2C, 3 × SPI, I2S, touch, RMT
Supply	5 V via USB/VIN, on-board 3.3 V regulator (AMS1117)
Logic level	3.3 V — inputs are NOT 5 V tolerant

Table 2: Key specifications of the ESP32 DevKit V1

3.5 Working Principle of Components

ACS712-5A

Current through a copper conductor inside the IC creates a magnetic field that a Hall sensor converts to voltage: Vout = 2.5 V + 0.185 V/A × I. The conductor is isolated from the sensing circuit (2.1 kV RMS), so the low-voltage side is safe. Because the output can reach 3.4 V, a 10 kΩ/20 kΩ divider scales it by 2/3 for the 3.3 V ESP32 ADC.

ZMPT101B

A small precision current transformer with a series resistor works as a voltage transformer. An LM358 op-amp amplifies the secondary signal and centres it at VCC/2. The output is an AC waveform proportional to mains voltage; the gain is set with a trimmer and fine-tuned in software.

Relay module

An opto-coupler drives a transistor that energises the relay coil. A LOW input switches the relay ON (active-LOW), and the contacts (10 A 250 V AC) connect or break the load's live wire.

I2C LCD

A PCF8574 I/O expander converts the two-wire I2C bus into the eight data/control lines of an HD44780 16x2 character LCD, saving GPIO pins.

4. Circuit Design

4.1 Circuit Diagram

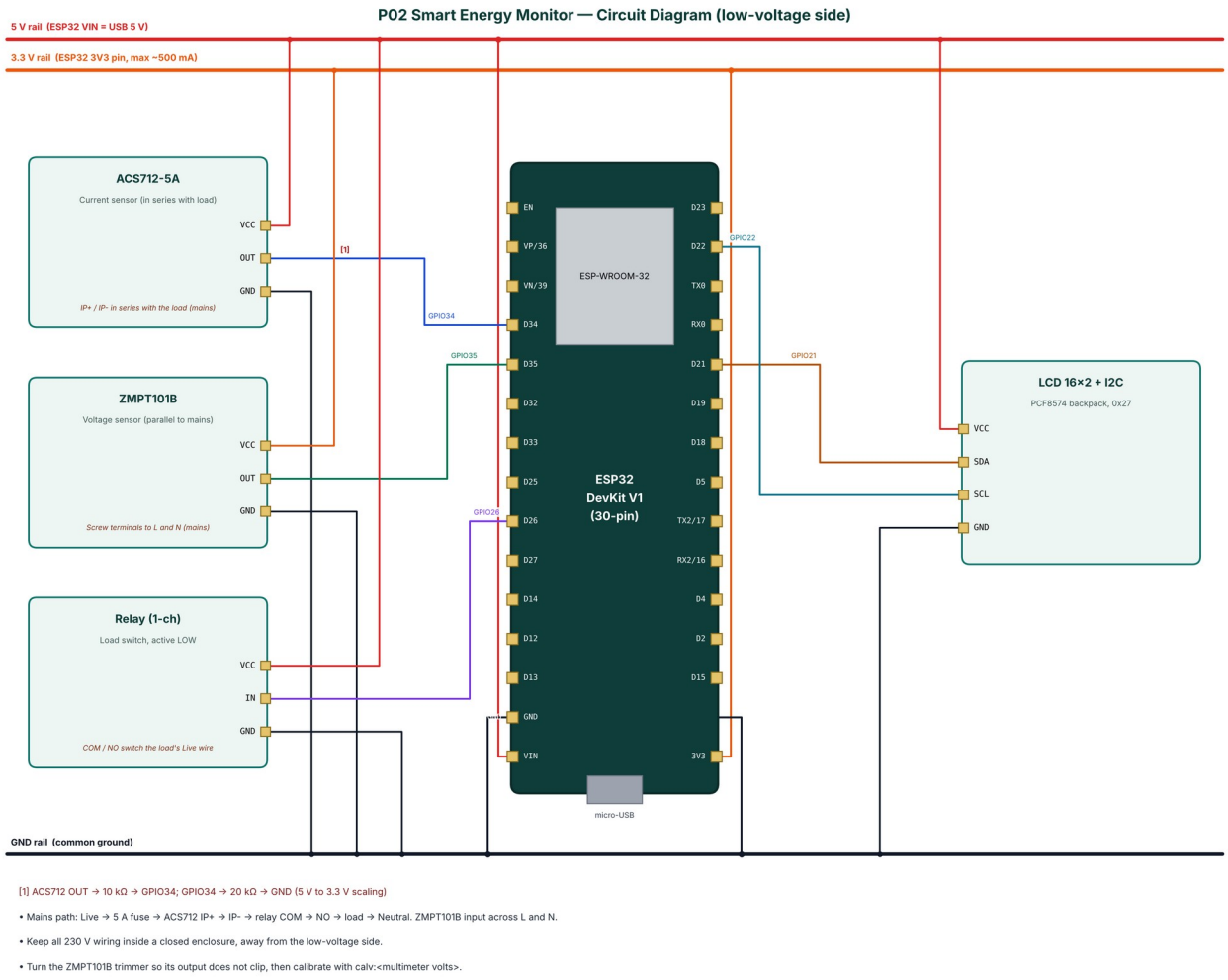


Figure 2: Circuit diagram of the Smart Energy Monitor

4.2 Pin Connections

Module	Module pin	ESP32 connection	Notes
ACS712-5A	VCC	5V rail (VIN)	
ACS712-5A	OUT	GPIO34	ACS712 OUT -> 10 kΩ -> GPIO34; GPIO34 -> 20 kΩ -> GND (5 V to 3.3 V scaling)
ACS712-5A	GND	GND	
ZMPT101B	VCC	3.3 V (3V3 pin)	
ZMPT101B	OUT	GPIO35	
ZMPT101B	GND	GND	
Relay (1-ch)	VCC	5V rail (VIN)	
Relay (1-ch)	IN	GPIO26	
Relay (1-ch)	GND	GND	
LCD 16x2 + I2C	VCC	5V rail (VIN)	

Module	Module pin	ESP32 connection	Notes
LCD 16x2 + I2C	SDA	GPIO21	
LCD 16x2 + I2C	SCL	GPIO22	
LCD 16x2 + I2C	GND	GND	

Table 3: Pin connection table

4.3 Interfacing and Power Notes

- Mains path: Live -> 5 A fuse -> ACS712 IP+ -> IP- -> relay COM -> NO -> load -> Neutral. ZMPT101B input across L and N.
- Keep all 230 V wiring inside a closed enclosure, away from the low-voltage side.
- Turn the ZMPT101B trimmer so its output does not clip, then calibrate with calv:<multimeter volts>.
- Only ADC1 pins (GPIO32-39) are used for analog inputs, because ADC2 cannot be read while Wi-Fi is active.
- Strapping pins GPIO0, 2, 12 and 15 are avoided for inputs so the board always boots normally.
- All grounds are common. 5 V signals are reduced to 3.3 V with resistor dividers before reaching the ESP32.

5. Data Flow Diagrams

Data flow diagrams (DFDs) show how data moves through the system without showing control logic or timing. External entities are drawn as rectangles, processes as numbered circles, data stores as open boxes and data flows as labelled arrows.

5.1 DFD Level 0 (Context Diagram)

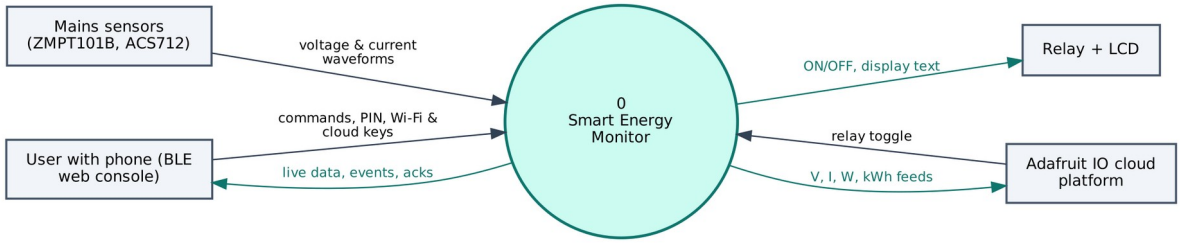


Figure 3: DFD Level 0 — context diagram

At level 0 the whole Smart Energy Monitor is a single process. It receives voltage & current waveforms from the Mains sensors (ZMPT101B, ACS712), commands and configuration from the user's phone, and commands from Adafruit IO. It sends ON/OFF, display text to the Relay + LCD, live data to the phone and V, I, W, kWh feeds to Adafruit IO.

5.2 DFD Level 1

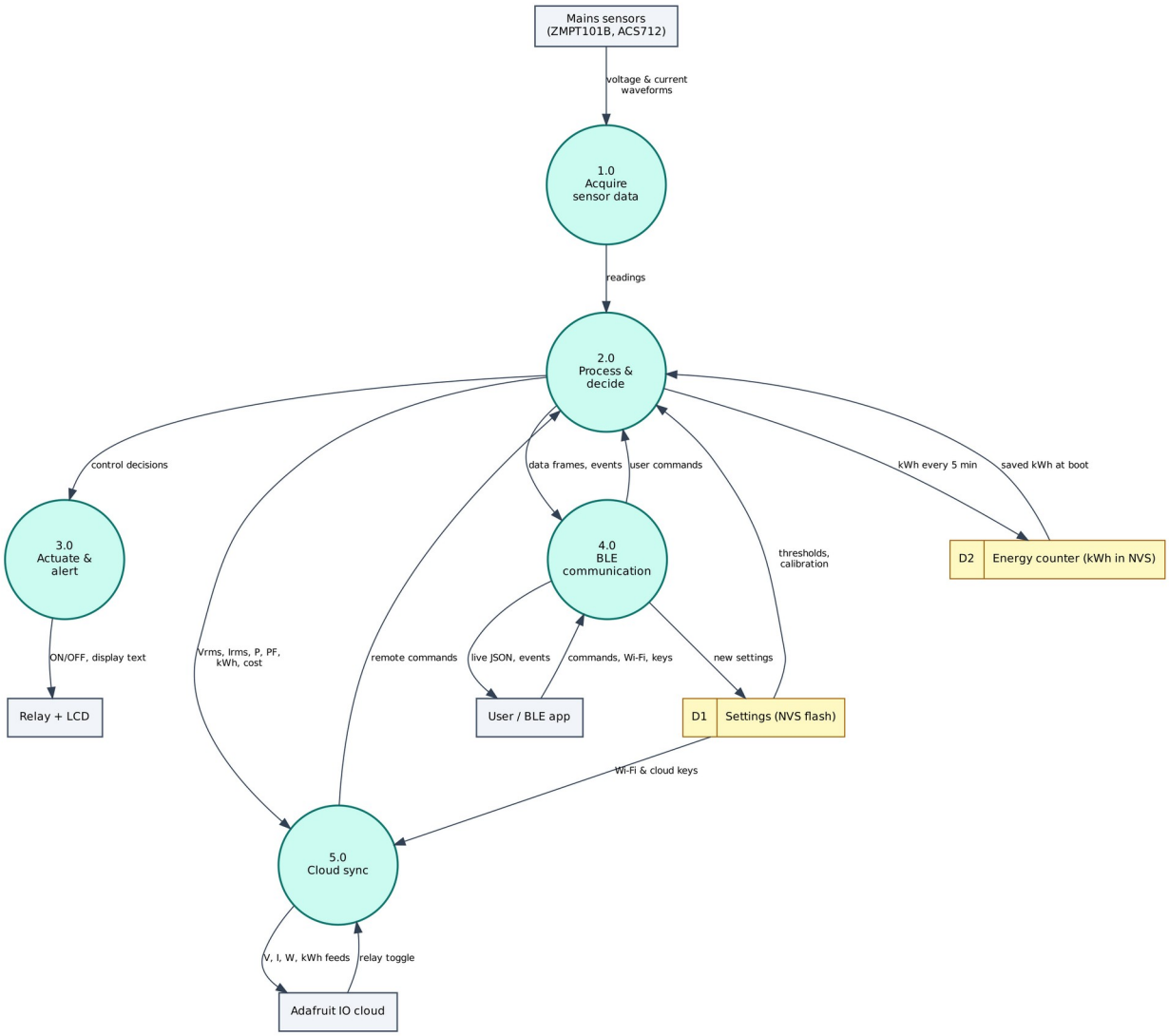


Figure 4: DFD Level 1 — main processes and data stores

Process / store	Description
1.0 Acquire sensor data	Reads voltage & current waveforms from the Mains sensors (ZMPT101B, ACS712); filters/averages the raw values.
2.0 Process & decide	Computes Vrms, Irms, P, PF, kWh, cost; applies thresholds and rules; generates events.
3.0 Actuate & alert	Drives the Relay + LCD (ON/OFF, display text).
4.0 BLE communication	Sends JSON frames and events to the phone; receives and authenticates commands.
5.0 Cloud sync	Sends V, I, W, kWh feeds to Adafruit IO; receives relay toggle.
D1 Settings (NVS)	Wi-Fi credentials, cloud keys, PIN, thresholds — survive reboots.
D2 Energy counter (kWh in NVS)	Stores kWh every 5 min; returns saved kWh at boot.

Table 4: Description of level-1 processes and data stores

5.3 Data Dictionary

Field	Meaning	Unit
v	RMS voltage	V
i	RMS current	A
w	Real power	W
va	Apparent power	VA
pf	Power factor	—
kwh	Energy	kWh
cost	Cost	₹
relay	Relay state	bool
cloud	MQTT connected	bool

Table 5: BLE data frame fields

Cloud object	Content / usage
p02-voltage	Voltage (V) — line chart / gauge
p02-current	Current (A) — gauge
p02-power	Real power (W) — line chart
p02-energy	Energy (kWh) — text block
p02-relay	Toggle block, ON/OFF values — remote load switch

Table 6: Adafruit IO data objects

6. Software Design and Implementation

6.1 Development Environment

Item	Details
IDE / build	Arduino IDE 2.x or arduino-cli; GitHub Actions builds firmware in the cloud
Board package	esp32 by Espressif Systems 3.x (tested with 3.3.12)
Board settings	ESP32 Dev Module, Partition Scheme: Minimal SPIFFS (1.9MB APP with OTA/190KB SPIFFS)
Common libraries	NimBLE-Arduino 2.x, ArduinoJson 7.x, PubSubClient 2.8
Project libraries	LiquidCrystal I2C (Frank de Brabander / johnrickman)
Phone app	Web Bluetooth console (Chrome on Android / desktop, Bluefy on iPhone); nRF Connect as alternative
Cloud	Adafruit IO (free tier)

Table 7: Software tools

> The default 1.2 MB app partition is too small for BLE + Wi-Fi + TLS. Select **Minimal SPIFFS (1.9MB APP)** in Tools → Partition Scheme, otherwise the upload fails with 'Sketch too big'.

6.2 Firmware Architecture

The firmware is split into three layers so that the ten projects share tested code:

- **IoTCore.h** (shared) — Wi-Fi station with auto-reconnect, NimBLE GATT server with the Nordic UART Service, line-based command parser shared by BLE and USB serial, PIN authentication, settings in NVS (Preferences), NTP time in IST, JSON helpers.
- **Cloud.h** (shared) — ThingSpeak HTTPS client, Adafruit IO MQTT/TLS client with a 28 messages/minute rate limiter, Supabase PostgREST client (insert, update, select).
- **P02_Smart_Energy_Monitor.ino** — sensor drivers, algorithms, automation rules and the project's own commands.

The main loop is non-blocking: every task is scheduled with millis() timers, BLE writes are queued from the BLE callback into a FreeRTOS queue and executed in loop(), so the BLE stack never waits for slow operations such as HTTPS requests.

6.3 Working of the System

Every second the firmware captures 2,000 sample pairs (voltage and current) in 200 ms, i.e. ten full 50 Hz cycles at a 10 kHz pair rate. The mean of each channel is the DC offset and is subtracted. Then Vrms = $\sqrt{(\Sigma v^2/N)}$, Irms = $\sqrt{(\Sigma i^2/N)}$ and real power P = $\Sigma(v \cdot i)/N$, each multiplied by its calibration factor. Apparent power S = Vrms·Irms and power factor PF = P/S.

Energy is integrated as E += P·Δt and stored in NVS every five minutes. Cost = kWh × tariff (default ₹7/kWh). If P exceeds the limit (default 1,000 W) for three consecutive seconds the relay opens and an 'overload' event is raised; the short delay avoids false trips from motor inrush current.

Four feeds (voltage, current, power, energy) are published to Adafruit IO every 15 seconds (16 messages per minute, under the 30/minute free limit). The firmware subscribes to the p02-relay feed, so a toggle

block on the dashboard switches the load. The LCD alternates between a V/A/W page and a PF/kWh/₹ page.

6.4 Flowchart

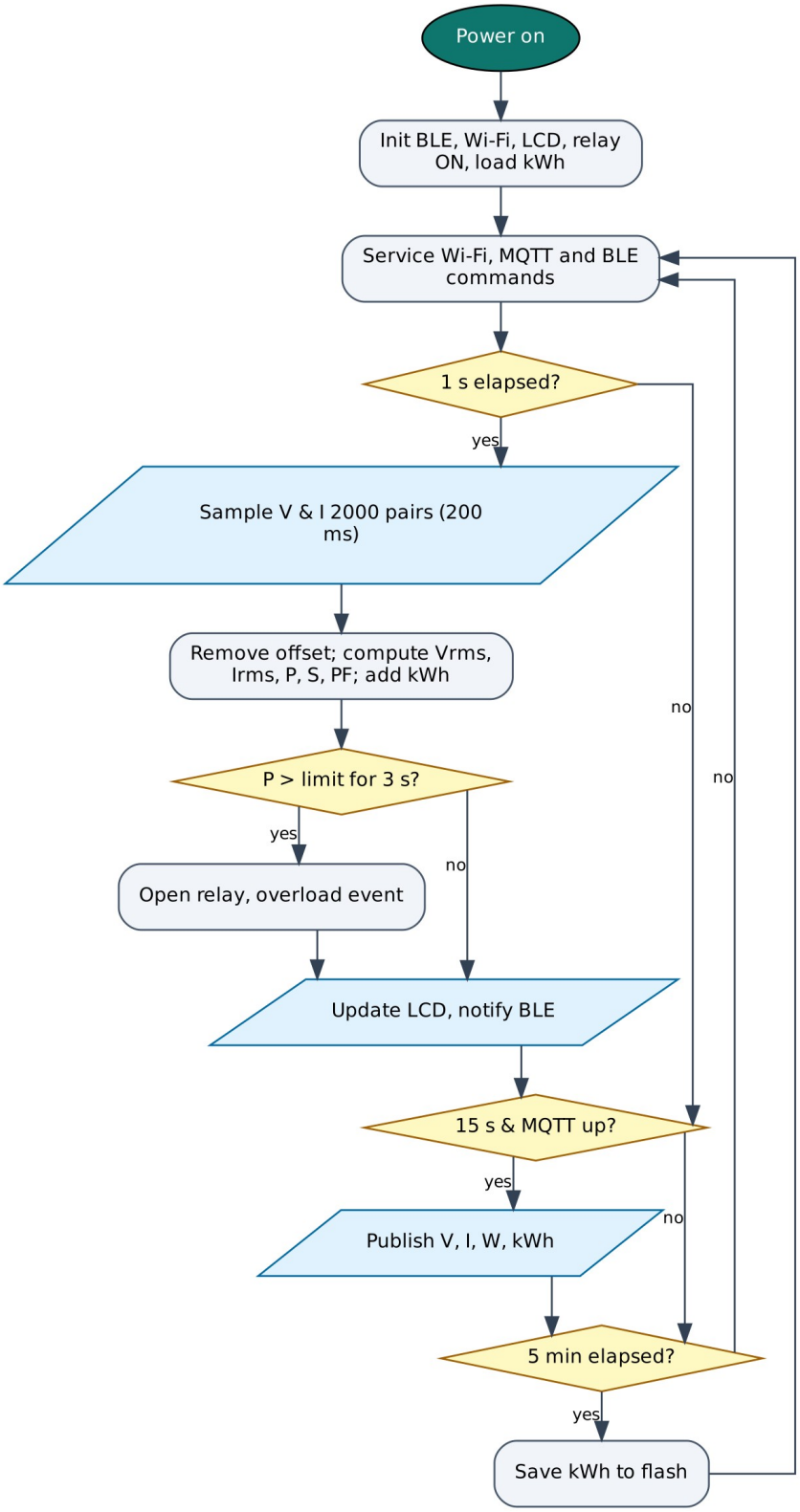


Figure 5: Firmware flowchart

6.5 BLE Communication Protocol

The device is a BLE peripheral named **IOT-P02-XXXX** (XXXX = last bytes of the MAC address). It exposes the Nordic UART Service, which most BLE apps understand:

Item	UUID	Properties
Service (NUS)	6E400001-B5A3-F393-E0A9-E50E24DCCA9E	Primary
RX — phone → ESP32	6E400002-B5A3-F393-E0A9-E50E24DCCA9E	Write, Write without response
TX — ESP32 → phone	6E400003-B5A3-F393-E0A9-E50E24DCCA9E	Notify, Read

Table 8: GATT service and characteristics

Commands are plain text lines (`command:argument`). Replies are one JSON object per line ending in a newline; long lines are split into MTU-sized notifications and re-joined by the app. Commands that change anything require `auth:<PIN>` first on each connection (default PIN 1234, change it with `set:pin=XXXX`).

Command	PIN needed	Action
auth:<PIN>	—	Unlock for this connection
status	No	Status JSON: Wi-Fi, IP, RSSI, heap, uptime, time
wifi:<SSID>,<password>	Yes	Save Wi-Fi credentials and connect
set:<key>=<value>	Yes	Save a setting (cloud keys, thresholds)
get:<key>	Yes	Read a setting (secrets masked)
reboot / forget	Yes	Restart / erase all settings
relay:on / relay:off	Yes	Switch the load
reset	Yes	Reset kWh counter
calv:<volts>	Yes	Calibrate voltage against a multimeter reading
cali:<amps>	Yes	Calibrate current against a clamp-meter reading

Table 9: BLE commands (common commands first, then project commands)

Example data frame sent by the device:

```
{"v":231.4,"i":0.412,"w":88.6,"va":95.3,"pf":0.93,"kwh":1.274,"cost":8.92,"relay":true,"cloud":true,"type":"data"}
```

6.6 Cloud Integration (Adafruit IO)

The device keeps a persistent MQTT connection to io.adafruit.com on port 8883 (TLS) using the account username and key. Values are published to `username/feeds/<feed-key>`, and control feeds are subscribed so dashboard changes arrive within about a second. A built-in limiter keeps the device below Adafruit IO's free limit of 30 data points per minute.

Object	Content
p02-voltage	Voltage (V) — line chart / gauge
p02-current	Current (A) — gauge
p02-power	Real power (W) — line chart
p02-energy	Energy (kWh) — text block
p02-relay	Toggle block, ON/OFF values — remote load switch

Table 10: Adafruit IO configuration

6.7 Configurable Settings

Key	Default	Meaning
aiouser / aiokey	—	Adafruit IO username and key
maxw	1000	Overload trip limit (W)
tariff	7	Electricity tariff (₹/kWh)
vcal / ical	0.70 / 0.00811	Calibration factors (set by calv/cali)
ssid / pass	—	Wi-Fi network (set with the wifi: command)
pin	1234	BLE PIN

Table 11: Settings stored in flash (change with set:key=value)

7. Setup and Deployment Guide

7.1 Hardware Assembly

1. Place the ESP32 DevKit across the centre gap of the breadboard so both pin rows are free.
2. Connect the ESP32 3V3 pin to one breadboard power rail (3.3 V), VIN to the other rail (5 V) and both GND pins to the ground rails.
3. Wire each module exactly as in the pin connection table (section 4.2), including the resistor dividers marked with [n] in the circuit diagram.
4. Check every connection against the circuit diagram before powering on. Make sure no 5 V signal reaches an ESP32 pin directly.
5. Connect the USB cable to a computer (for flashing) or a 5 V adapter (for normal use).

7.2 Flashing the Firmware

Option A — Arduino IDE

1. Install Arduino IDE 2.x from arduino.cc.
2. File → Preferences → Additional boards manager URLs: `https://espressif.github.io/arduino-esp32/package_esp32_index.json`.
3. Tools → Board → Boards Manager: install **esp32 by Espressif Systems** (version 3.x).
4. Sketch → Include Library → Manage Libraries: install **NimBLE-Arduino** (2.x), **ArduinoJson** (7.x), **PubSubClient**, **LiquidCrystal I2C (Frank de Brabander / johnrickman)**.
5. Download the repository and open the folder **P02_Smart_Energy_Monitor** (it already contains IoTCore.h, Cloud.h and config.h).
6. Tools: Board **ESP32 Dev Module**, Partition Scheme **Minimal SPIFFS (1.9MB APP with OTA/190KB SPIFFS)**, Upload Speed 921600, and the correct COM port.
7. Click Upload. If it stays at 'Connecting...', hold the BOOT button on the ESP32 until upload starts.
8. Open Serial Monitor at 115200 baud and type `status` to check the device.

Option B — no software installation (GitHub Actions + browser flasher)

1. Open the repository https://github.com/rohitkukreja705/esp32-iot-projects → **Actions** → latest **Build firmware** run.
2. Download the artifact **firmware-P02_Smart_Energy_Monitor** and unzip it to get `P02_Smart_Energy_Monitor.ino.merged.bin`.
3. Open **https://espressif.github.io/esptool-js/** in Chrome or Edge on a computer, click **Connect** and select the ESP32's serial port.
4. Set Flash Address **0x0**, choose the merged .bin file and click **Program**. Press the EN (reset) button when done.

7.3 Cloud Setup (Adafruit IO)

1. Go to **io.adafruit.com** and create a free account.
2. Click the yellow **key icon (My Key / Adafruit IO Key)** and copy your **Username** and **Active Key**.

3. In the BLE web console open **Setup**, unlock and save them in `aiouser` and `aiokey` (commands: `set:aiouser=NAME`, `set:aiokey=aio_xxxx`).
4. Power the device: feeds are created automatically on the first publish. You can also create them first in **Feeds → New Feed** using the exact keys in the table below.
5. Open **Dashboards → New Dashboard**, then **+ (Create New Block)**. Use Toggle blocks for switch feeds with button-on text `ON` and button-off text `OFF` (or `AUTO`/`MANUAL` for mode feeds), Gauge blocks for levels and Line Chart blocks for history.
6. Free tier: 30 data points per minute, 10 feeds, 5 dashboards and 30 days of history per account. Projects on the same account share these limits, so delete feeds of projects you are not running or use separate accounts.

Field / feed / table	Content
p02-voltage	Voltage (V) — line chart / gauge
p02-current	Current (A) — gauge
p02-power	Real power (W) — line chart
p02-energy	Energy (kWh) — text block
p02-relay	Toggle block, ON/OFF values — remote load switch

Table 12: Items to create in Adafruit IO

7.4 BLE App Setup (Web Bluetooth Console)

No app installation is needed. The console is a web page hosted free on GitHub Pages; Chrome talks to the ESP32 directly through the Web Bluetooth API.

1. On an Android phone turn on **Bluetooth** and **Location** (Android requires location for BLE scanning).
2. Open **https://rohitkukreja705.github.io/esp32-iot-projects/?p=P02** in **Chrome**. (On iPhone use the free **Bluefy** browser; on a laptop use Chrome or Edge.)
3. Tap **Connect** and choose **IOT-P02-XXXX** in the pop-up list. The dot turns green and the Live tab starts updating.
4. Open **Setup**, enter the PIN (default **1234**) and tap **Unlock**.
5. Enter your 2.4 GHz Wi-Fi name and password and tap **Save Wi-Fi**. After a few seconds the status line shows the IP address.
6. Enter the Adafruit IO keys from section 7.3 and tap **Save** next to each.
7. Use the **Control** tab for commands and the **Log** tab to see every event and acknowledgement.

Alternative: nRF Connect app

1. Install **nRF Connect for Mobile** (Nordic Semiconductor, free) and scan.
2. Connect to **IOT-P02-XXXX** and open **Nordic UART Service**.
3. Enable notifications on the **TX** characteristic (6E400003…).
4. Write UTF-8 text to the **RX** characteristic (6E400002…), e.g. `auth:1234`, then `status`.

> To host the console yourself: in the GitHub repository open Settings → Pages, choose 'Deploy from a branch', branch main, folder /docs. Web Bluetooth only works on HTTPS pages, which GitHub Pages provides.

7.5 First-Run Checklist

- Serial Monitor shows '[BLE] advertising as IOT-P02-XXXX'.
- Status in the console shows wifi: true and an IP address.
- Data appears in Adafruit IO within a minute.
- All sensor values in the Live tab are plausible (no '—').

8. Testing and Results

The following test cases verify each function. Record the observed result and mark Pass/Fail while testing your build.

#	Test	Procedure	Expected result	Observed
T1	No-load reading	Power the meter with the load switched off	Voltage ≈ mains value, current 0 A, power 0 W	
T2	Calibration	Measure mains with a multimeter, send calv:<value>	Displayed voltage matches the multimeter within ±2 %	
T3	Resistive load	Connect a 100 W bulb or 1 kW heater	Power ≈ rated value, PF ≈ 0.95-1.0	
T4	Inductive load	Connect a table fan	PF clearly below 1 (typ. 0.7-0.9)	
T5	Overload trip	Set maxw=50 and connect a 100 W load	Relay opens after ~3 s; 'overload' event shown	
T6	Remote switch	Toggle p02-relay on the Adafruit IO dashboard	Relay follows within ~1 s; BLE event logged	
T7	Power-cut retention	Note kWh, unplug for 1 min, re-plug	kWh resumes from the last saved value (≤ 5 min loss)	

Table 13: Test cases

8.1 Expected Output

A typical data frame received in the console's Log tab looks like this (values will differ):

```
{"v":231.4,"i":0.412,"w":88.6,"va":95.3,"pf":0.93,"kwh":1.274,"cost":8.92,"relay":true,"cloud":true,"type":"data"}
```

Serial Monitor (115200 baud) prints the same lines along with connection messages such as `[WiFi] connected`, `[BLE] connected` and the HTTP or MQTT result of each cloud upload, which helps in troubleshooting.

8.2 Troubleshooting

Symptom	Likely cause and fix
Device not in the Connect list	Bluetooth/Location off on the phone, or the page is not opened in Chrome over HTTPS.
'Sketch too big' on upload	Select Partition Scheme 'Minimal SPIFFS (1.9MB APP with OTA)'.
Wi-Fi never connects	ESP32 supports only 2.4 GHz networks; check the SSID/password (case-sensitive).
Settings refused	Send auth:<PIN> first; the unlock is per BLE connection.
Sensor shows null / —	Check wiring and supply voltage; see the Serial Monitor for 'not found' messages.
Board resets randomly	Weak USB supply or servo/relay current; use a 5 V 2 A adapter and add a 470 µF capacitor.
No data in Adafruit IO	Wrong username/key, rate limit exceeded or 10-feed limit reached.

Table 14: Troubleshooting guide

9. Discussion

9.1 Advantages

- Measures real power and power factor, not just current.
- Automatic overload protection with remote and local reset.
- Calibratable in the field from a phone.
- Energy and cost history in a free cloud dashboard.
- Low-voltage side is galvanically isolated from mains by both sensors.

9.2 Limitations

- ACS712-5A resolution is about 25 mA after scaling, so loads under ~5 W are not measured well.
- ESP32 ADC is non-linear near 0 V and 3.3 V; accuracy is about ±2-3 % after calibration.
- Single channel, single phase only.
- Adafruit IO free tier keeps data for 30 days.

9.3 Safety Precautions

- 230 V AC can kill. Build only under supervision of a person experienced with mains wiring.
- Use a closed plastic enclosure, a 5 A fuse in the live line and proper screw terminals; no breadboard on the mains side.
- Never touch the board or sensors while the mains plug is connected; program and test the low-voltage side first.
- The ACS712-5A must not carry more than 5 A continuously; size the fuse accordingly.

9.4 Security Considerations

- Secrets (Wi-Fi password, cloud keys) are stored in the ESP32's NVS flash, not in source code.
- BLE commands that change the device require a PIN; change the default PIN after setup.
- Cloud traffic is TLS-encrypted. Certificate verification is disabled (setInsecure) for simplicity; for production, pin the server's root certificate with setCACert().
- Use a dedicated cloud account/project for the device and rotate keys if the device is lost.

9.5 Future Scope

- Use a dedicated metering IC (HLW8012, PZEM-004T) for revenue-grade accuracy.
- Add scheduling (e.g. geyser only 6-7 AM) and monthly bill prediction.
- Monitor several circuits with split-core CT sensors (SCT-013).
- Detect appliance type from its power signature (NILM).

10. Conclusion

The Smart Energy Monitor measures real power and power factor with two inexpensive isolated sensors and turns the ESP32 into a connected, protective smart plug. Users can see consumption and cost on an LCD, a phone or a free cloud dashboard, switch the load from anywhere and rely on automatic overload protection. With its calibration routine it achieves accuracy suitable for monitoring and energy-saving decisions.

All design goals listed in Chapter 1 are met with low-cost parts available in the Indian market, free cloud services and open-source software. The shared firmware core and the single Web Bluetooth console make the project easy to reproduce and extend.

References

1. Allegro MicroSystems, ACS712 Fully Integrated Hall-Effect Current Sensor datasheet.
2. ZMPT101B Current-type Voltage Transformer datasheet, Qingxian Zeming Langxi Electronic.
3. OpenEnergyMonitor, 'AC Power Theory - Arduino maths', learn.openenergymonitor.org.
4. Adafruit Industries, Adafruit IO MQTT API documentation, io.adafruit.com/api/docs/mqtt.html.
5. Espressif Systems, ESP32 Series Datasheet and ESP32 Technical Reference Manual, espressif.com.
6. Espressif Systems, Arduino-ESP32 documentation, docs.espressif.com/projects/arduino-esp32.
7. H2zero, NimBLE-Arduino library documentation, github.com/h2zero/NimBLE-Arduino.
8. Bluetooth SIG, Bluetooth Core Specification 4.2 — Generic Attribute Profile (GATT).
9. W3C Web Bluetooth Community Group, Web Bluetooth specification, webbluetoothcg.github.io/web-bluetooth.
10. B. Blanchon, ArduinoJson 7 documentation, arduinojson.org.
11. Adafruit Industries, Adafruit IO documentation and MQTT API, io.adafruit.com/api/docs.

Appendix A: Project Source Code

The complete code is in the repository folder **P02_Smart_Energy_Monitor** (https://github.com/rohitkukreja705/esp32-iot-projects).

config.h

```
#pragma once
// ===== P02 Smart Energy Monitor — configuration =====
#define WIFI_SSID     ""
#define WIFI_PASS     ""
#define DEVICE_PIN    "1234"
#define AIO_USERNAME  ""     // Adafruit IO username (or set:aiouser=...)
#define AIO_KEY       ""     // Adafruit IO key      (or set:aiokey=...)

// ----- pins -----
#define PIN_CURRENT   34     // ACS712-5A OUT through 10k/20k divider (ADC1)
#define PIN_VOLTAGE   35     // ZMPT101B OUT (module powered from 3.3 V)
#define PIN_RELAY     26     // relay module IN
#define RELAY_ACTIVE_LOW true
// LCD 16x2 with PCF8574 backpack: SDA = 21, SCL = 22, address 0x27 (try 0x3F)
#define LCD_ADDR      0x27

// ----- calibration defaults (fine-tune with calv:<V> and cali:<A>) -----
#define DEF_V_PER_MV  0.70f   // mains volts per ADC millivolt RMS (ZMPT pot dependent)
#define DEF_A_PER_MV  0.00811f// 1 / (185 mV/A * 2/3 divider) = 0.00811 A per mV
#define NOISE_AMPS    0.05f   // readings below this are treated as 0 A

// ----- behaviour -----
#define SAMPLE_MS        1000    // measure + BLE + LCD
#define CLOUD_MS         15000   // Adafruit IO publish (4 feeds -> 16 msg/min)
#define DEF_MAX_WATTS    1000    // overload trip (set:maxw=...)
#define DEF_TARIFF       7.0     // Rs per kWh   (set:tariff=...)
```

P02_Smart_Energy_Monitor.ino

```
/*
 * P02 — Smart Energy Monitor with Overload Protection
 * ===================================================
 * Peripherals: ACS712-5A current sensor, ZMPT101B voltage sensor,
 *              16x2 I2C LCD, 1-channel relay
 * Wi-Fi  : Adafruit IO over MQTT — voltage, current, power, energy feeds and
 *          a "relay" toggle feed for remote ON/OFF from the dashboard
 * BLE    : live V / I / W / PF / kWh / cost, relay control, calibration
 * Extras : true RMS + real power + power factor from simultaneous sampling,
 *          kWh counter saved to flash, overload auto-trip
 *
 * !!! SAFETY: the ZMPT101B and the relay connect to 230 V AC mains. Build it
 * inside a closed plastic enclosure, use a fuse, never touch the board while
 * plugged in, and get help from someone experienced with mains wiring. !!!
 *
 * Libraries: NimBLE-Arduino 2.x, ArduinoJson 7, PubSubClient, LiquidCrystal_I2C
 */
#include "config.h"
#include "IoTCore.h"
#include "Cloud.h"
#include <Wire.h>
#include <LiquidCrystal_I2C.h>

LiquidCrystal_I2C lcd(LCD_ADDR, 16, 2);
cloud::AdafruitIO aio;
```

```
const char* SUB_FEEDS[] = { "p02-relay" };

struct Meter {
  float vrms = 0, irms = 0, realW = 0, va = 0, pf = 0;
  double kwh = 0;
} m;

float vPerMv, aPerMv, maxWatts, tariff;
bool relayOn = true;
int overCount = 0;
uint32_t lastSample = 0, lastCloud = 0, lastSave = 0, lastMeasure = 0;
int lcdPage = 0;
// raw RMS of the last measurement (mV at the ADC) - used for calibration
float rawVmv = 0, rawImv = 0;

void loadSettings() {
  vPerMv   = iot::settingF("vcal", DEF_V_PER_MV);
  aPerMv   = iot::settingF("ical", DEF_A_PER_MV);
  maxWatts = iot::settingF("maxw", DEF_MAX_WATTS);
  tariff   = iot::settingF("tariff", DEF_TARIFF);
}

void setRelay(bool on, const char* source) {
  relayOn = on;
  digitalWrite(PIN_RELAY, (on ^ RELAY_ACTIVE_LOW) ? HIGH : LOW);
  iot::sendEvent("relay", String(on ? "ON" : "OFF") + " by " + source);
  aio.publish("p02-relay", on ? "ON" : "OFF");
}

/*
 * Sample both channels ~2000 times across 200 ms (10 mains cycles at 50 Hz).
 * The DC offset (mean) is removed, then:
 *   Vrms = sqrt(mean(v^2)), Irms = sqrt(mean(i^2)), P = mean(v*i)
 */
void measure() {
  const int N = 2000;
  static int16_t vs[N], is[N];
  uint32_t t0 = micros();
  for (int k = 0; k < N; k++) {
    vs[k] = analogRead(PIN_VOLTAGE);
    is[k] = analogRead(PIN_CURRENT);
    while (micros() - t0 < (uint32_t)(k + 1) * 100) { }   // 10 kHz pair rate
  }
  double vMean = 0, iMean = 0;
  for (int k = 0; k < N; k++) { vMean += vs[k]; iMean += is[k]; }
  vMean /= N; iMean /= N;
  double vv = 0, ii = 0, vi = 0;
  for (int k = 0; k < N; k++) {
    double v = vs[k] - vMean, i = is[k] - iMean;
    vv += v * v; ii += i * i; vi += v * i;
  }
  const double mvPerCount = 3300.0 / 4095.0;              // approx. with 11 dB attenuation
  rawVmv = sqrt(vv / N) * mvPerCount;
  rawImv = sqrt(ii / N) * mvPerCount;
  m.vrms = rawVmv * vPerMv;
  m.irms = rawImv * aPerMv;
  if (m.vrms < 20) m.vrms = 0;                            // mains not present
  if (m.irms < NOISE_AMPS) m.irms = 0;
  double p = (vi / N) * mvPerCount * mvPerCount * vPerMv * aPerMv;
  m.va = m.vrms * m.irms;
  m.realW = (m.irms == 0) ? 0 : fabs(p);
  if (m.realW > m.va) m.realW = m.va;
  m.pf = m.va > 1 ? m.realW / m.va : 0;
```

```
  uint32_t now = millis();
  if (lastMeasure) m.kwh += m.realW * (now - lastMeasure) / 3600000.0 / 1000.0;
  lastMeasure = now;
}

void protect() {
  if (relayOn && m.realW > maxWatts) {
    if (++overCount >= 3) {                          // 3 s above the limit
      setRelay(false, "overload");
      iot::sendEvent("overload", "Load " + String(m.realW, 0) + " W > limit " + String(maxWatts, 0) + "
W");
      overCount = 0;
    }
  } else overCount = 0;
}

void showLcd() {
  char l1[17], l2[17];
  if (lcdPage < 3) {
    snprintf(l1, 17, "%5.1fV %6.3fA", m.vrms, m.irms);
    snprintf(l2, 17, "%6.1fW %s %s", m.realW, relayOn ? "ON " : "OFF", iot::wifiOK() ? "W" : "-");
  } else {
    snprintf(l1, 17, "PF %.2f %6.0fVA", m.pf, m.va);
    snprintf(l2, 17, "%.3fkWh Rs%.1f", m.kwh, m.kwh * tariff);
  }
  lcd.setCursor(0, 0); lcd.print(l1); for (int i = strlen(l1); i < 16; i++) lcd.print(' ');
  lcd.setCursor(0, 1); lcd.print(l2); for (int i = strlen(l2); i < 16; i++) lcd.print(' ');
  lcdPage = (lcdPage + 1) % 5;                       // 3 s page 1, 2 s page 2
}

void sendBle() {
  JsonDocument d;
  d["v"] = iot::rnd(m.vrms); d["i"] = iot::rnd(m.irms, 3);
  d["w"] = iot::rnd(m.realW); d["va"] = iot::rnd(m.va);
  d["pf"] = iot::rnd(m.pf, 2); d["kwh"] = iot::rnd(m.kwh, 3);
  d["cost"] = iot::rnd(m.kwh * tariff, 2);
  d["relay"] = relayOn; d["cloud"] = aio.connected();
  iot::sendData(d);
}

void onFeed(const String& feed, const String& value) {
  Serial.printf("[AIO] %s = %s\n", feed.c_str(), value.c_str());
  if (feed == "p02-relay") {
    bool on = value.equalsIgnoreCase("ON") || value == "1";
    if (on != relayOn) setRelay(on, "cloud");
  }
}

bool onCommand(const String& cmd, const String& arg, bool authed) {
  if (cmd == "setting") { loadSettings(); return true; }
  if (cmd == "relay" || cmd == "reset" || cmd == "calv" || cmd == "cali") {
    if (!authed) { iot::emitAck(cmd, false, "send auth:<PIN> first"); return true; }
    if (cmd == "relay") {
      bool on = arg == "on" ? true : arg == "off" ? false : !relayOn;
      setRelay(on, "BLE"); iot::emitAck(cmd, true, on ? "ON" : "OFF");
    } else if (cmd == "reset") {
      m.kwh = 0; iot::saveSetting("kwh", "0"); iot::emitAck(cmd, true, "energy reset");
    } else if (cmd == "calv") {                // calv:<voltage shown by a multimeter>
      if (rawVmv < 1) { iot::emitAck(cmd, false, "no voltage signal"); return true; }
      vPerMv = arg.toFloat() / rawVmv; iot::saveSetting("vcal", String(vPerMv, 5));
      iot::emitAck(cmd, true, "vcal=" + String(vPerMv, 5));
    } else if (cmd == "cali") {                // cali:<amps shown by a clamp meter>
```

```
      if (rawImv < 1) { iot::emitAck(cmd, false, "no current signal"); return true; }
      aPerMv = arg.toFloat() / rawImv; iot::saveSetting("ical", String(aPerMv, 6));
      iot::emitAck(cmd, true, "ical=" + String(aPerMv, 6));
    }
    return true;
  }
  return false;
}

void setup() {
  pinMode(PIN_RELAY, OUTPUT);
  analogSetAttenuation(ADC_11db);
  iot::begin("P02", onCommand);
  loadSettings();
  m.kwh = iot::settingF("kwh", 0);
  setRelay(true, "boot");
  Wire.begin(21, 22);
  lcd.init(); lcd.backlight();
  lcd.print("Energy Monitor"); lcd.setCursor(0, 1); lcd.print("starting...");
  aio.begin(onFeed, SUB_FEEDS, 1);
}

void loop() {
  iot::loop();
  aio.loop();
  uint32_t now = millis();
  if (now - lastSample >= SAMPLE_MS) {
    lastSample = now;
    measure();
    protect();
    showLcd();
    sendBle();
  }
  if (now - lastCloud >= CLOUD_MS && aio.connected()) {
    lastCloud = now;
    aio.publish("p02-voltage", m.vrms, 1);
    aio.publish("p02-current", m.irms, 3);
    aio.publish("p02-power", m.realW, 1);
    aio.publish("p02-energy", (float)m.kwh, 3);
  }
  if (now - lastSave >= 300000) {                          // save kWh every 5 min
    lastSave = now;
    iot::saveSetting("kwh", String(m.kwh, 4));
  }
}
```

Appendix B: Shared Library Code

These two headers are identical in every project folder (master copies in /common).

IoTCore.h

```
/*
 * IoTCore.h  —  shared runtime for all ESP32 IoT projects in this repo
 * ---------------------------------------------------------------------
 *  - Wi-Fi station with automatic reconnect (credentials stored in flash)
 *  - BLE server using the Nordic UART Service (NUS) layout, so the device
 *    works with the Web Bluetooth dashboard in /docs AND with generic apps
 *    such as "nRF Connect" or "Serial Bluetooth Terminal"
 *  - Line-based command protocol shared by BLE and USB Serial
 *  - Persistent settings (Preferences / NVS): Wi-Fi, cloud keys, PIN, thresholds
 *  - NTP time sync (IST) for timestamps
 *
 *  Wire protocol
 *  -------------
 *  Phone -> ESP32 (write to RX characteristic, text):
 *      auth:1234              unlock control + settings for this connection
 *      status                 device replies with a status JSON line
 *      set:key=value          store a setting (Wi-Fi, cloud key, threshold…)
 *      get:key                read back a setting (secrets are masked)
 *      wifi:SSID,PASSWORD     shortcut for set:ssid + set:pass + reconnect
 *      reboot                 restart the board
 *      <anything else>        forwarded to the project's own command handler
 *  ESP32 -> Phone (notify on TX characteristic):
 *      one JSON object per line, terminated by '\n'
 *      {"type":"data",...}  {"type":"status",...}  {"type":"ack",...}
 *      {"type":"event",...} {"type":"err",...}
 *  Lines longer than one BLE packet are split into chunks; the receiver
 *  joins chunks until it sees '\n'.
 *
 *  Library: NimBLE-Arduino 2.x (lighter than Bluedroid, leaves more RAM for TLS)
 *  Board:   ESP32 Dev Module, Partition Scheme "Minimal SPIFFS (1.9MB APP with OTA)"
 */
#pragma once

#include <Arduino.h>
#include <WiFi.h>
#include <Preferences.h>
#include <NimBLEDevice.h>
#include <ArduinoJson.h>
#include <time.h>

#ifndef DEVICE_PIN
#define DEVICE_PIN "1234"
#endif
#ifndef WIFI_SSID
#define WIFI_SSID ""
#endif
#ifndef WIFI_PASS
#define WIFI_PASS ""
#endif

namespace iot {

// Nordic UART Service UUIDs (widely supported by BLE terminal apps)
static const char* NUS_SERVICE = "6E400001-B5A3-F393-E0A9-E50E24DCCA9E";
static const char* NUS_RX      = "6E400002-B5A3-F393-E0A9-E50E24DCCA9E";  // phone writes here
static const char* NUS_TX      = "6E400003-B5A3-F393-E0A9-E50E24DCCA9E";  // ESP32 notifies here
```

```
// Project command handler: return true if the command was recognised.
using CommandHandler = bool (*)(const String& cmd, const String& arg, bool authed);

// ---------------------------------------------------------------- state
struct CmdMsg { char text[256]; bool fromBle; };

static Preferences        prefs;
static NimBLEServer*      bleServer   = nullptr;
static NimBLECharacteristic* txChr    = nullptr;
static QueueHandle_t      cmdQueue    = nullptr;
static CommandHandler     userHandler = nullptr;
static String             deviceName;
static String             projectId;
static volatile uint16_t  bleMtu      = 23;
static volatile bool      bleAuthed   = false;   // reset on every new connection
static bool               serialAuthed = true;   // USB = physical access, trusted
static bool               timeSynced  = false;
static uint32_t           lastWifiTry = 0;
static String             rxLine;                // assembles multi-packet writes

// ---------------------------------------------------------------- settings
inline String setting(const char* key, const char* def = "") {
  return prefs.isKey(key) ? prefs.getString(key, def) : String(def);
}
inline void saveSetting(const char* key, const String& v) { prefs.putString(key, v); }
inline float settingF(const char* key, float def) {
  String s = setting(key, "");
  return s.length() ? s.toFloat() : def;
}

// ---------------------------------------------------------------- BLE
inline bool bleConnected() { return bleServer && bleServer->getConnectedCount() > 0; }

// Send one line (adds '\n'), split to fit the negotiated MTU.
inline void bleSend(const String& line) {
  if (!bleConnected() || !txChr) return;
  String msg = line + "\n";
  size_t chunk = (bleMtu > 23 ? bleMtu : 23) - 3;
  for (size_t i = 0; i < msg.length(); i += chunk) {
    size_t n = min(chunk, msg.length() - i);
    txChr->setValue((const uint8_t*)msg.c_str() + i, n);
    txChr->notify();
    if (msg.length() > chunk) delay(8);   // give the stack time between chunks
  }
}

// Send to BLE and mirror to USB Serial.
inline void emit(const String& line) {
  Serial.println(line);
  bleSend(line);
}

inline void emitAck(const String& cmd, bool ok, const String& msg = "") {
  JsonDocument d;
  d["type"] = ok ? "ack" : "err";
  d["cmd"] = cmd;
  if (msg.length()) d["msg"] = msg;
  String s; serializeJson(d, s); emit(s);
}

inline void enqueue(const String& s, bool fromBle) {
  if (!cmdQueue || s.length() == 0) return;
```

```
  CmdMsg m; memset(&m, 0, sizeof(m));
  strncpy(m.text, s.c_str(), sizeof(m.text) - 1);
  m.fromBle = fromBle;
  xQueueSend(cmdQueue, &m, 0);
}

class ServerCB : public NimBLEServerCallbacks {
  void onConnect(NimBLEServer* s, NimBLEConnInfo& info) override {
    bleAuthed = false;
    rxLine = "";
    Serial.printf("[BLE] connected %s\n", info.getAddress().toString().c_str());
  }
  void onDisconnect(NimBLEServer* s, NimBLEConnInfo& info, int reason) override {
    bleAuthed = false;
    bleMtu = 23;
    Serial.printf("[BLE] disconnected (reason %d), advertising again\n", reason);
    // NimBLE restarts advertising automatically (advertiseOnDisconnect = true)
  }
  void onMTUChange(uint16_t mtu, NimBLEConnInfo& info) override {
    bleMtu = mtu;
    Serial.printf("[BLE] MTU %u\n", mtu);
  }
};

class RxCB : public NimBLECharacteristicCallbacks {
  void onWrite(NimBLECharacteristic* c, NimBLEConnInfo& info) override {
    std::string v = c->getValue();
    // Accept either a single write without newline, or several writes
    // that together form lines terminated by '\n'.
    for (char ch : v) {
      if (ch == '\r') continue;
      if (ch == '\n') { enqueue(rxLine, true); rxLine = ""; }
      else if (rxLine.length() < 250) rxLine += ch;
    }
    if (v.find('\n') == std::string::npos && rxLine.length() && rxLine.length() == v.length()) {
      enqueue(rxLine, true);  // plain one-shot write (e.g. typed in nRF Connect)
      rxLine = "";
    }
  }
};

inline void bleBegin() {
  uint8_t mac[6]; WiFi.macAddress(mac);
  char suffix[6]; snprintf(suffix, sizeof(suffix), "%02X%02X", mac[4], mac[5]);
  deviceName = "IOT-" + projectId + "-" + suffix;

  NimBLEDevice::init(deviceName.c_str());
  NimBLEDevice::setMTU(247);
  bleServer = NimBLEDevice::createServer();
  bleServer->setCallbacks(new ServerCB());

  NimBLEService* svc = bleServer->createService(NUS_SERVICE);
  txChr = svc->createCharacteristic(NUS_TX, NIMBLE_PROPERTY::NOTIFY | NIMBLE_PROPERTY::READ);
  NimBLECharacteristic* rx = svc->createCharacteristic(
      NUS_RX, NIMBLE_PROPERTY::WRITE | NIMBLE_PROPERTY::WRITE_NR, 512);
  rx->setCallbacks(new RxCB());
  // (NimBLE 2.x starts services automatically when advertising begins)

  // 128-bit UUID goes in the advertisement, the name in the scan response
  // (both together would not fit in one 31-byte packet).
  NimBLEAdvertisementData adv, scan;
  adv.setFlags(BLE_HS_ADV_F_DISC_GEN | BLE_HS_ADV_F_BREDR_UNSUP);
  adv.setCompleteServices(NimBLEUUID(NUS_SERVICE));
```

```
  scan.setName(deviceName.c_str());
  NimBLEAdvertising* a = NimBLEDevice::getAdvertising();
  a->setAdvertisementData(adv);
  a->setScanResponseData(scan);
  a->start();
  Serial.printf("[BLE] advertising as %s\n", deviceName.c_str());
}

// ---------------------------------------------------------------- Wi-Fi
inline bool wifiOK() { return WiFi.status() == WL_CONNECTED; }

inline void wifiConnect() {
  String ssid = setting("ssid", WIFI_SSID);
  String pass = setting("pass", WIFI_PASS);
  if (ssid.isEmpty()) {
    Serial.println("[WiFi] no SSID yet - send  wifi:SSID,PASSWORD  over BLE");
    return;
  }
  Serial.printf("[WiFi] connecting to %s\n", ssid.c_str());
  WiFi.disconnect();
  WiFi.begin(ssid.c_str(), pass.c_str());
  lastWifiTry = millis();
}

inline void wifiLoop() {
  static bool wasUp = false;
  bool up = wifiOK();
  if (up && !wasUp) {
    Serial.printf("[WiFi] connected, IP %s, RSSI %d\n",
                  WiFi.localIP().toString().c_str(), WiFi.RSSI());
    configTzTime("IST-5:30", "pool.ntp.org", "time.google.com");
  }
  if (!up && millis() - lastWifiTry > 20000) wifiConnect();   // retry every 20 s
  wasUp = up;
  if (up && !timeSynced && time(nullptr) > 1700000000) {
    timeSynced = true;
    Serial.println("[NTP] time synced");
  }
}

// ISO-8601 local time, or uptime if NTP has not synced yet.
inline String timestamp() {
  time_t now = time(nullptr);
  if (now < 1700000000) return "uptime+" + String(millis() / 1000) + "s";
  struct tm t; localtime_r(&now, &t);
  char buf[32]; strftime(buf, sizeof(buf), "%Y-%m-%dT%H:%M:%S+05:30", &t);
  return String(buf);
}

// ------------------------------------------------------- built-in commands
inline bool isSecretKey(const String& k) {
  return k == "pass" || k.endsWith("key") || k == "pin";
}

inline void sendStatus() {
  JsonDocument d;
  d["type"] = "status";
  d["dev"] = deviceName;
  d["wifi"] = wifiOK();
  d["ssid"] = setting("ssid", WIFI_SSID);
  d["rssi"] = wifiOK() ? WiFi.RSSI() : 0;
  d["ip"] = wifiOK() ? WiFi.localIP().toString() : "";
  d["auth"] = bleAuthed;
```

```
  d["heap"] = ESP.getFreeHeap();
  d["up"] = millis() / 1000;
  d["time"] = timestamp();
  String s; serializeJson(d, s); emit(s);
}

inline void handleLine(const String& raw, bool fromBle) {
  String line = raw; line.trim();
  if (line.isEmpty()) return;
  int c = line.indexOf(':');
  String cmd = (c < 0 ? line : line.substring(0, c));
  String arg = (c < 0 ? "" : line.substring(c + 1));
  cmd.trim(); cmd.toLowerCase();
  bool authed = fromBle ? bleAuthed : serialAuthed;

  if (cmd == "auth") {
    bool ok = arg == setting("pin", DEVICE_PIN);
    if (fromBle) bleAuthed = ok;
    emitAck(cmd, ok, ok ? "unlocked" : "wrong PIN");
    return;
  }
  if (cmd == "status") { sendStatus(); return; }
  if (cmd == "ping")   { emitAck(cmd, true, "pong"); return; }

  // Everything below changes the device, so it needs the PIN over BLE.
  if (cmd == "set" || cmd == "wifi" || cmd == "reboot" || cmd == "get" || cmd == "forget") {
    if (!authed) { emitAck(cmd, false, "send auth:<PIN> first"); return; }
    if (cmd == "set") {
      int e = arg.indexOf('=');
      if (e <= 0) { emitAck(cmd, false, "use set:key=value"); return; }
      String k = arg.substring(0, e); k.trim();
      String v = arg.substring(e + 1); v.trim();
      if (k.length() > 15) { emitAck(cmd, false, "key too long"); return; }
      saveSetting(k.c_str(), v);
      emitAck(cmd, true, k + " saved");
      if (k == "ssid" || k == "pass") wifiConnect();
      // let the project react to changed thresholds etc.
      if (userHandler) userHandler("setting", k, true);
      return;
    }
    if (cmd == "get") {
      String v = setting(arg.c_str(), "");
      if (isSecretKey(arg) && v.length()) v = "****" + v.substring(max(0, (int)v.length() - 3));
      emitAck(cmd, true, arg + "=" + v);
      return;
    }
    if (cmd == "wifi") {
      int k = arg.indexOf(',');
      String ssid = k < 0 ? arg : arg.substring(0, k);
      String pass = k < 0 ? "" : arg.substring(k + 1);
      saveSetting("ssid", ssid); saveSetting("pass", pass);
      emitAck(cmd, true, "connecting to " + ssid);
      wifiConnect();
      return;
    }
    if (cmd == "forget") { prefs.clear(); emitAck(cmd, true, "all settings erased"); return; }
    if (cmd == "reboot") { emitAck(cmd, true, "rebooting"); delay(300); ESP.restart(); }
  }

  if (userHandler && userHandler(cmd, arg, authed)) return;
  emitAck(cmd, false, "unknown command");
}
```

```
// ------------------------------------------------------------ entry points
// Call once from setup().  id = short project id, e.g. "P01".
inline void begin(const char* id, CommandHandler handler) {
  Serial.begin(115200);
  delay(200);
  projectId = id;
  userHandler = handler;
  prefs.begin("iot", false);
  cmdQueue = xQueueCreate(8, sizeof(CmdMsg));
  WiFi.mode(WIFI_STA);
  WiFi.setAutoReconnect(true);
  // NOTE: Wi-Fi modem sleep must stay ON while BLE is running (ESP32 coexistence).
  bleBegin();
  wifiConnect();
  Serial.println("[CORE] ready - type 'status' here or connect over BLE");
}

// Call every loop() iteration.
inline void loop() {
  wifiLoop();
  // USB serial commands (handy while developing)
  static String serialLine;
  while (Serial.available()) {
    char ch = Serial.read();
    if (ch == '\n' || ch == '\r') { if (serialLine.length()) enqueue(serialLine, false); serialLine =
""; }
    else if (serialLine.length() < 250) serialLine += ch;
  }
  CmdMsg m;
  while (cmdQueue && xQueueReceive(cmdQueue, &m, 0) == pdTRUE) handleLine(String(m.text), m.fromBle);
}

// Round for compact JSON. NaN stays NaN, which ArduinoJson writes as null.
inline float rnd(float v, int dp = 1) {
  if (isnan(v)) return v;
  float m = powf(10, dp);
  return roundf(v * m) / m;
}

// Helper: build and send a {"type":"data",...} line from a filled document.
inline void sendData(JsonDocument& d) {
  d["type"] = "data";
  String s; serializeJson(d, s); emit(s);
}

inline void sendEvent(const String& name, const String& msg) {
  JsonDocument d;
  d["type"] = "event"; d["name"] = name; d["msg"] = msg; d["time"] = timestamp();
  String s; serializeJson(d, s); emit(s);
}

}  // namespace iot
```

Cloud.h

```
/*
 * Cloud.h  —  free-tier cloud helpers used by the projects
 * ------------------------------------------------------------
 *  ThingSpeak   : HTTPS REST "update" (free tier: 1 update / 15 s per channel)
 *  Adafruit IO  : MQTT over TLS (free tier: 30 data points / minute, 10 feeds)
 *  Supabase     : PostgREST over HTTPS (free tier: 500 MB Postgres)
 *
```

```
 *  Keys are read from settings (stored over BLE with set:key=value) so no
 *  secret has to be hard-coded or committed to GitHub:
 *     tskey    ThingSpeak Write API Key
 *     aiouser  Adafruit IO username        aiokey  Adafruit IO key
 *     sburl    https://<ref>.supabase.co   sbkey   Supabase publishable/anon key
 *
 *  TLS note: certificate checking is disabled with setInsecure() to keep the
 *  projects beginner-friendly and independent of CA rotations. Traffic is
 *  still encrypted. For production, pin the root CA with setCACert().
 */
#pragma once

#include <WiFiClientSecure.h>
#include <HTTPClient.h>
#include <PubSubClient.h>
#include "IoTCore.h"

// Optional compile-time defaults (normally left empty and set over BLE)
#ifndef TS_WRITE_KEY
#define TS_WRITE_KEY ""
#endif
#ifndef AIO_USERNAME
#define AIO_USERNAME ""
#endif
#ifndef AIO_KEY
#define AIO_KEY ""
#endif
#ifndef SUPABASE_URL
#define SUPABASE_URL ""
#endif
#ifndef SUPABASE_KEY
#define SUPABASE_KEY ""
#endif

namespace cloud {

// ============================================================== ThingSpeak
// vals[i] -> field(i+1). Pass NAN to skip a field. Returns true on success.
inline bool thingSpeakUpdate(const float* vals, int n, const String& status = "") {
  if (!iot::wifiOK()) return false;
  String key = iot::setting("tskey", TS_WRITE_KEY);
  if (key.isEmpty()) { Serial.println("[TS] no tskey set"); return false; }
  String url = "https://api.thingspeak.com/update?api_key=" + key;
  for (int i = 0; i < n && i < 8; i++)
    if (!isnan(vals[i])) url += "&field" + String(i + 1) + "=" + String(vals[i], 2);
  if (status.length()) {
    String st = status; st.replace(" ", "%20");
    url += "&status=" + st;
  }
  WiFiClientSecure client; client.setInsecure();
  HTTPClient http; http.setTimeout(8000);
  if (!http.begin(client, url)) return false;
  int code = http.GET();
  String body = http.getString();
  http.end();
  bool ok = code == 200 && body.toInt() > 0;   // ThingSpeak returns the entry id, 0 = rejected
  Serial.printf("[TS] HTTP %d entry=%s\n", code, body.c_str());
  return ok;
}

// ============================================================== Supabase
// Generic PostgREST call. method: GET/POST/PATCH. path e.g. "/rest/v1/readings"
inline int supabaseRequest(const char* method, const String& path, const String& body,
```

```
                           String* response = nullptr, const char* prefer = "return=minimal") {
  if (!iot::wifiOK()) return -1;
  String base = iot::setting("sburl", SUPABASE_URL);
  String key  = iot::setting("sbkey", SUPABASE_KEY);
  if (base.isEmpty() || key.isEmpty()) { Serial.println("[SB] sburl/sbkey not set"); return -2; }
  if (base.endsWith("/")) base.remove(base.length() - 1);
  WiFiClientSecure client; client.setInsecure();
  HTTPClient http; http.setTimeout(8000);
  if (!http.begin(client, base + path)) return -3;
  http.addHeader("apikey", key);
  http.addHeader("Authorization", "Bearer " + key);
  http.addHeader("Content-Type", "application/json");
  if (prefer) http.addHeader("Prefer", prefer);
  int code = http.sendRequest(method, body);
  if (response) *response = http.getString();
  else if (code >= 300) Serial.printf("[SB] %s\n", http.getString().c_str());
  http.end();
  Serial.printf("[SB] %s %s -> %d\n", method, path.c_str(), code);
  return code;
}

inline bool supabaseInsert(const char* table, const String& json) {
  int c = supabaseRequest("POST", String("/rest/v1/") + table, json);
  return c >= 200 && c < 300;
}

// filter is a PostgREST filter, e.g. "id=eq.1"
inline bool supabaseUpdate(const char* table, const String& filter, const String& json) {
  int c = supabaseRequest("PATCH", String("/rest/v1/") + table + "?" + filter, json);
  return c >= 200 && c < 300;
}

// query e.g. "select=*&id=eq.1"  -> JSON array text in out
inline bool supabaseSelect(const char* table, const String& query, String& out) {
  int c = supabaseRequest("GET", String("/rest/v1/") + table + "?" + query, "", &out, nullptr);
  return c == 200;
}

// ============================================================ Adafruit IO
// MQTT wrapper with a simple rate limiter to respect the 30 msg/min limit.
using FeedCallback = void (*)(const String& feed, const String& value);

class AdafruitIO {
 public:
  // requestLast = true asks Adafruit IO for each feed's last value on connect
  // (useful for switches changed while offline; unsafe for things like pumps)
  void begin(FeedCallback cb, const char* const* subFeeds, int nSub, bool requestLast = true) {
    cb_ = cb; subs_ = subFeeds; nSub_ = nSub; requestLast_ = requestLast;
    tls_.setInsecure();
    mqtt_.setClient(tls_);
    mqtt_.setServer("io.adafruit.com", 8883);
    mqtt_.setBufferSize(512);
    mqtt_.setKeepAlive(60);
    mqtt_.setCallback([](char* topic, byte* payload, unsigned int len) {
      String t(topic), v;
      for (unsigned int i = 0; i < len; i++) v += (char)payload[i];
      int p = t.lastIndexOf('/');
      String feed = t.substring(p + 1);
      if (instance()->cb_) instance()->cb_(feed, v);
    });
    instance() = this;
  }
```

```
  void loop() {
    if (!iot::wifiOK()) return;
    if (!mqtt_.connected()) {
      if (millis() - lastTry_ < 10000) return;
      lastTry_ = millis();
      user_ = iot::setting("aiouser", AIO_USERNAME);
      String key = iot::setting("aiokey", AIO_KEY);
      if (user_.isEmpty() || key.isEmpty()) { Serial.println("[AIO] aiouser/aiokey not set"); return; }
      String id = "esp32-" + String((uint32_t)ESP.getEfuseMac(), HEX);
      Serial.println("[AIO] connecting...");
      if (mqtt_.connect(id.c_str(), user_.c_str(), key.c_str())) {
        Serial.println("[AIO] connected");
        for (int i = 0; i < nSub_; i++) {
          mqtt_.subscribe(topic(subs_[i]).c_str());
          if (requestLast_) mqtt_.publish((topic(subs_[i]) + "/get").c_str(), "");
        }
        // subscribe to throttle/error topics so we learn about rate limits
        mqtt_.subscribe((user_ + "/throttle").c_str());
      } else {
        Serial.printf("[AIO] failed, state %d\n", mqtt_.state());
      }
    }
    mqtt_.loop();
  }

  bool connected() { return mqtt_.connected(); }

  // Returns false if not connected or if the per-minute budget is used up.
  bool publish(const char* feed, const String& value) {
    if (!mqtt_.connected()) return false;
    uint32_t now = millis();
    if (now - windowStart_ > 60000) { windowStart_ = now; sent_ = 0; }
    if (sent_ >= 28) { Serial.println("[AIO] rate budget used, skipping"); return false; }
    sent_++;
    return mqtt_.publish(topic(feed).c_str(), value.c_str());
  }
  bool publish(const char* feed, float v, int dp = 2) { return publish(feed, String(v, dp)); }

 private:
  String topic(const char* feed) { return user_ + "/feeds/" + feed; }
  static AdafruitIO*& instance() { static AdafruitIO* p = nullptr; return p; }

  WiFiClientSecure tls_;
  PubSubClient mqtt_;
  FeedCallback cb_ = nullptr;
  const char* const* subs_ = nullptr;
  int nSub_ = 0;
  bool requestLast_ = true;
  String user_;
  uint32_t lastTry_ = 0, windowStart_ = 0;
  int sent_ = 0;
};

}  // namespace cloud
```